

TRANSFORMING ATTENTION: GENERATING MULTI-HEAD ATTENTION FROM LEARNED GROUP ORBITS

Shahil Shaik¹ Aayush Rai¹ Anshul Nayak¹ Jonathon M. Smereka² Yue Wang¹ *

ABSTRACT

Multi-head attention (MHA) represents diverse interactions by assigning independent query, key, and value projections to every head. Although flexible, this construction imposes no structure on how heads are related and makes both projection parameters and autoregressive key-value (KV) state scale with the number of heads. We introduce group-transformed multi-head attention (GT-MHA), which instead generates head-specific interactions from a smaller set of learned bases. A learned matrix-group transformation maps each base query-key scoring geometry to a head-specific geometry, while a separate transformation produces the corresponding value representation. A single base provides the strongest sharing, whereas multiple bases define distinct transformation orbits and offer greater representational flexibility. We characterize the induced bilinear scoring forms, showing that heads within an orbit vary their interactions inside fixed learned query and key subspaces, while different orbits may provide different subspaces. This construction makes the base-projection parameters and persistent KV-cache scale with the number of bases rather than the number of generated heads, while retaining the full collection of attention maps and the standard multi-head interface. We implement the transformations using the exact matrix exponential and efficient first- and second-order approximations. On a 12-layer TinyStoriesV2 language model, second-order GT-MHA attains a validation perplexity of 1.3416, compared with 1.3425 for MHA, while reducing total parameters by 18.1%, attention parameters by 54.7%, and persistent KV-cache memory by 75%.

1 INTRODUCTION

Attention determines which elements of a representation interact and how information flows among them. Multi-head attention (MHA) enriches this operation by learning several interaction patterns in parallel, allowing different heads to emphasize different relationships (Vaswani et al., 2017). In the standard construction, each head receives its own query, key, and value projections. Although these heads are optimized jointly, their parameterization imposes no structure on how their query-key compatibility functions or value representations are related. This design is highly flexible, but it assumes that useful head diversity must be supported by fully independent projection parameters. Empirical evidence that many learned heads are redundant or can be removed with little loss suggests that this assumption may be stronger than necessary (Michel et al., 2019; Voita et al., 2019). We therefore ask: *can a small set of learned bases generate a larger collection of expressive, head-specific interactions through structured transformations?*

Motivated by this possibility, we introduce Group-Transformed Multi-Head Attention (GT-MHA), a structured parameterization that generates attention heads from one or more shared bases. Rather than learning every head from scratch, GT-MHA first learns a small collection of base query, key, and value projections. Each head is associated with one base and specializes it along the scoring and value paths. On the scoring path, a learned element of a matrix group transforms the base query-key scoring geometry into a head-specific geometry. Similarly, a separate learned transformation produces the head-specific representation along the value path. Heads associated with the same base

^{*}¹Department of Mechanical Engineering, Clemson University, Clemson, SC 29631, USA
{shahils, arai, anshuln, yue6}@clemson.edu

²U.S. Army CCDC Ground Vehicle Systems Center, Warren, MI 48397, USA
jonathon.m.smereka.civ@army.mil

therefore share an underlying structure while retaining distinct scoring functions and value representations.

The number of bases controls the strength and structure of this sharing. With a single base, all heads belong to one transformation orbit, yielding the most compact parameterization and the strongest coupling among heads. Introducing multiple bases partitions the heads into distinct orbit families, each defined by its own learned projections. This allows different families to capture different query and key subspaces while preserving structured variation among heads within each family. Our analysis makes this distinction precise: transformations within an orbit modify the bilinear interactions inside a fixed pair of learned query and key subspaces, whereas different orbits may provide different subspaces. The number of bases therefore offers a direct trade-off between parameter sharing and representational flexibility.

Beyond organizing head diversity, this structure provides concrete parameter and inference-memory savings. Because the base projections are learned once per base rather than once per head, their parameter count scales with the number of bases instead of the total number of heads. During autoregressive decoding, keys and values can likewise be retained at the base level, so the persistent KV-cache scales with the number of bases rather than the number of generated heads. GT-MHA nevertheless produces the full collection of distinct attention maps and preserves the standard multi-head input-output interface. We parameterize the transformations using shared Lie-algebra generators and head-specific coordinates, evaluating both the exact matrix exponential and efficient first- and second-order Taylor-series approximations.

We evaluate GT-MHA in autoregressive language modeling under controlled training settings. On TinyStoriesV2, a 12-layer model using the second-order transformation approximation attains a validation perplexity of 1.3416, compared with 1.3425 for standard MHA, while reducing total model parameters by 18.1%, attention parameters by 54.7%, and persistent KV-cache memory by 75%. These results provide initial evidence that structured transformations of shared bases can preserve the predictive performance of independently parameterized heads while substantially reducing their parameter and inference-memory requirements. [Will be modified once we have complete results]

In summary, this work makes three contributions:

- We introduce GT-MHA, a structured attention parameterization that generates head-specific scoring geometries and value representations from one or more learned bases through matrix-group transformations.
- We characterize the resulting families of bilinear scoring forms, clarify the distinction between variation within an orbit and across multiple orbits, and analyze the associated parameter, computational, and KV-cache costs. We further develop exact and approximate implementations of the transformations.
- We evaluate GT-MHA in autoregressive language modeling and show that it can preserve the predictive performance of standard MHA while substantially reducing attention parameters, total model size, and persistent KV-cache memory.

2 RELATED WORK

2.1 STRUCTURED SHARING IN MULTI-HEAD ATTENTION

Empirical studies have shown that attention heads are not uniformly important: many heads can be pruned with little loss, while a smaller subset learns specialized functions (Michel et al., 2019; Voita et al., 2019). Complementary theory identifies a rank bottleneck induced by the finite query-key dimension of each head (Bhojanapalli et al., 2020). These findings motivate parameterizations that retain multiple attention maps while reducing redundant head-specific structure.

Several methods share or compress query, key, and value representations. Multi-query attention (MQA) shares one key-value pair across all query heads (Shazeer, 2019), while grouped-query attention (GQA) interpolates between MQA and MHA by assigning one key-value pair to each group of query heads (Ainslie et al., 2023). Decoupled-head attention learns layer-dependent groupings for key and value heads when converting pretrained MHA checkpoints (Chen et al., 2024), and multi-head latent attention compresses the cached key-value state through a shared low-rank latent

representation (DeepSeek-AI, 2024). These approaches primarily target autoregressive memory and bandwidth by reducing or compressing the number of stored key–value representations. GT-MHA instead retains H attention maps while generating their scoring geometries and value representations from C transformed bases; its base-level cache reduction follows from this parameterization.

The methods most closely related to GT-MHA introduce structured sharing across attention heads. Collaborative MHA learns common query–key features together with head-specific mixing coefficients (Cordonnier et al., 2020). Multiple-head embedding attention uses one seed projection for each of queries, keys, and values, then differentiates heads through additive or elementwise multiplicative embedding vectors (Xue & Aletras, 2023). FiSHformer constructs local attention matrices from a finite admixture of learned global attention matrices (Nguyen et al., 2022). GT-MHA instead imposes sharing through structured transformations: an element of a matrix group transforms a base query–key scoring geometry into a head-specific one, while a separate transformation produces the corresponding head-specific value representation. Thus, heads within a base follow a learned transformation orbit rather than being produced by mixing global attention matrices or by coordinatewise modulation of shared projections. Multiple bases extend this construction to a union of orbit families with distinct learned subspaces.

2.2 GROUP-BASED NEURAL PARAMETERIZATIONS

Group-equivariant neural networks use group actions to encode known transformations of a data domain. LieConv extends group convolutions to continuous data and broad classes of Lie groups (Finzi et al., 2020). Related attention architectures impose equivariance through group-invariant positional structure or by lifting inputs to functions on a group: group-equivariant stand-alone self-attention targets visual symmetries (Romero & Cordonnier, 2021), while LieTransformer constructs self-attention equivariant to Lie groups and their discrete subgroups (Hutchinson et al., 2021).

GT-MHA uses group structure for a different purpose. The group acts on learned attention parameters rather than on input tokens, spatial coordinates, or feature fields. Consequently, the construction neither assumes a corresponding symmetry of the data nor guarantees equivariance of the model’s input–output map. Instead, Lie-algebraic coordinates and their induced transformations organize parameter sharing across heads, producing structured families of query–key scoring geometries and value representations from learned bases.

3 BACKGROUND

3.1 MULTI-HEAD ATTENTION AS LEARNED SCORING GEOMETRY

Attention can be interpreted as a context-dependent message-passing operator over token representations. For each query token, the attention mechanism induces a data-dependent distribution over key tokens through learned pairwise compatibility scores, and uses this distribution to aggregate the corresponding value representations. The scoring function therefore defines the geometry under which tokens are judged to be similar, relevant, or informative to one another. Throughout this work, we use the term *scoring geometry* to mean this learned compatibility structure; it need not induce a metric, since the corresponding bilinear form may be asymmetric or indefinite.

Given an input sequence $X \in \mathbb{R}^{T \times d_{\text{model}}}$, where T denotes the sequence length and d_{model} is the model dimension, a standard attention head parameterizes this scoring function by projecting each token representation into query, key, and value spaces:

$$Q_h = XW_Q^{(h)}, \quad K_h = XW_K^{(h)}, \quad V_h = XW_V^{(h)},$$

where $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)} \in \mathbb{R}^{d_{\text{model}} \times d_h}$ are learned head-specific projection matrices, d_h is the head dimension, and $h \in \{1, \dots, H\}$ indexes the H attention heads. Each head forms pairwise query–key scores, normalizes them into attention weights, and uses these weights to aggregate the value representations:

$$\text{Attn}_h(X) = \text{softmax} \left(\frac{Q_h K_h^\top}{\sqrt{d_h}} \right) V_h.$$

The head outputs are then concatenated and projected back to the model dimension:

$$\text{MHA}(X) = \text{Concat}(\text{Attn}_1(X), \dots, \text{Attn}_H(X)) W_O,$$

where $W_O \in \mathbb{R}^{Hd_h \times d_{\text{model}}}$ is the output projection matrix.

This formulation highlights that each head defines its own bilinear query-key compatibility geometry through the head-specific projections $W_Q^{(h)}$ and $W_K^{(h)}$. More precisely, for two token representations $x_i, x_j \in \mathbb{R}^{d_{\text{model}}}$, define the effective scoring matrix of head h as

$$B_h = W_Q^{(h)} W_K^{(h)\top} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}.$$

The scaled compatibility score can then be written as

$$s_h(x_i, x_j) = \frac{x_i^\top B_h x_j}{\sqrt{d_h}}.$$

Because B_h factors through the d_h -dimensional head space, $\text{rank}(B_h) \leq d_h$. Consequently, each head defines a bilinear scoring geometry of rank at most d_h , determined jointly by its query and key projections.

Standard multi-head attention obtains diversity by parameterizing the projection factors of each head independently, without imposing explicit shared structure on the resulting scoring matrices B_h . Under the common fixed-width parameterization $d_{\text{model}} = Hd_h$, increasing the number of heads partitions the same projection dimension into more, narrower heads, leaving the total query, key, and value parameter count unchanged. At the level of the scoring parameterization, what changes is its granularity: the model contains more separately parameterized head geometries, with no explicit mechanism for capturing structure shared among them. This observation motivates our central question: *can diverse head-specific scoring geometries be generated as structured transformations of a smaller set of shared bases, rather than parameterized independently?*

3.2 GROUP ACTIONS AND TRANSFORMATION ORBITS

The preceding subsection interprets each attention head as a bilinear scoring geometry. To relate these geometries through transformations of shared bases, we use group actions and their orbits. A group specifies a family of composable and reversible transformations, an action specifies how those transformations modify a scoring geometry, and an orbit collects the geometries reachable from a given base.

Formally, a group G is a set equipped with an associative composition law $(g_1, g_2) \mapsto g_1 g_2$, an identity element e , and an inverse g^{-1} for every $g \in G$. An action of G on a space $\mathcal{S}$ is a map $\Phi : G \times \mathcal{S} \rightarrow \mathcal{S}$ satisfying

$$\Phi(e, s) = s, \quad \Phi(g_1, \Phi(g_2, s)) = \Phi(g_1 g_2, s)$$

for all $g_1, g_2 \in G$ and $s \in \mathcal{S}$. We write $\Phi(g, s) = g \cdot s$. The orbit of $s \in \mathcal{S}$ is

$$\mathcal{O}(s) = \{g \cdot s : g \in G\},$$

the set of objects reachable from s through transformations in G .

Among groups, Lie groups describe smoothly parameterized transformation families. A Lie group is both a group and a smooth manifold, with smooth multiplication and inversion. For a matrix Lie group $G \subseteq \text{GL}(d_h, \mathbb{R})$, the tangent space at the identity $T_e G$, denoted by $\mathfrak{g}$, is its Lie algebra and provides a linear space of infinitesimal transformation directions. The exponential map $\exp : \mathfrak{g} \rightarrow G$ maps algebra elements to finite group transformations, but need not be surjective.

GT-MHA takes $\mathcal{S}$ to be a space of bilinear query-key scoring geometries. Let $s^{(0)} \in \mathcal{S}$ denote a learned base geometry. Each learned head-specific element $g_h \in G$ selects a geometry on its orbit:

$$s_h = g_h \cdot s^{(0)} \in \mathcal{O}(s^{(0)}).$$

Using multiple learned bases yields a union of such orbits and allows the degree of sharing across heads to be varied. Thus, rather than parameterizing every query-key geometry independently, GT-MHA generates head geometries by transforming a smaller set of learned bases.

Crucially, G acts on $\mathcal{S}$, the scoring-parameter space, rather than on the input tokens or data domain. GT-MHA therefore neither posits a G -symmetry of the data nor imposes a G -invariance or G -equivariance constraint on the model’s input-output mapping. The action instead serves as a controlled parameter-sharing mechanism across attention heads. Section 4 specifies the corresponding matrix action and its parameterization.

4 GROUP-TRANSFORMED MULTI-HEAD ATTENTION(GT-MHA)

GT-MHA realizes the parameter-space action introduced in Section 3 by generating head-specific query–key scoring geometries from one or more learned bases. We first introduce a Lie-algebraic parameterization that maps compact head-specific coordinates to invertible transformations of the head space. We then use these transformations to construct single- and multi-orbit variants, which provide different degrees of sharing across attention heads. The value pathway is parameterized separately, as it determines the content being aggregated rather than the query–key compatibility geometry. [\[one basic image of the idea here.\]](#)

4.1 LIE-ALGEBRAIC PARAMETERIZATION OF HEAD TRANSFORMATIONS

GT-MHA constructs its head-specific transformations from a learned generator bank shared across heads. Let $\{E_\ell\}_{\ell=1}^p \subset \mathbb{R}^{d_h \times d_h}$ denote the generators, and let $\theta_h \in \mathbb{R}^p$ denote the head-specific logits. We define the normalized mixture weights π_h , the head-specific Lie-algebra element A_h , and the corresponding transformation M_h by

$$\pi_h = \text{softmax}(\theta_h), \quad A_h = \beta \sum_{\ell=1}^p \pi_{h,\ell} E_\ell, \quad M_h = \exp(A_h),$$

Here, $\pi_{h,\ell} \geq 0$ and $\sum_{\ell=1}^p \pi_{h,\ell} = 1$, so π_h determines the contribution of each shared generator to A_h , while $\beta > 0$ controls the overall transformation scale. In other words, the generator bank defines transformation directions common to all heads, while θ_h determines how those directions are combined for head h .

The generators E_ℓ , and hence every head-specific Lie algebra element A_h , belong to the full matrix Lie algebra $\mathfrak{gl}(d_h, \mathbb{R}) = \mathbb{R}^{d_h \times d_h}$. The matrix exponential maps these elements to the positive-determinant general linear group

$$\text{GL}^+(d_h, \mathbb{R}) = \{M \in \mathbb{R}^{d_h \times d_h} : \det(M) > 0\}.$$

Because $M_h = \exp(A_h)$, it satisfies $\det(M_h) = \exp(\text{tr}(A_h)) > 0$ and is therefore an invertible element of $\text{GL}^+(d_h, \mathbb{R})$ for every value of the learned parameters. The exponential parameterization thus enforces invertibility by construction, without requiring constrained optimization. [can add reference showing improvement over some other work](#) Section 4.2 shows how M_h transforms a shared base geometry into the scoring function of head h .

4.2 SINGLE-ORBIT GT-MHA

We first consider the single-orbit form of GT-MHA, in which all H attention heads are generated from one learned base query–key geometry. Shared query and key projections define this base, while the transformation M_h from Section 4.1 determines the scoring geometry of head h within the common d_h -dimensional head space. Thus, head diversity is expressed through M_h , rather than through independently learned query–key projection pairs as in standard MHA. [image describing single orbit here](#)

Let $W_Q^{(0)}, W_K^{(0)} \in \mathbb{R}^{d_{\text{model}} \times d_h}$ denote the shared base query and key projections. Given an input sequence $X \in \mathbb{R}^{T \times d_{\text{model}}}$, the corresponding base query and key representations are

$$Q^{(0)} = XW_Q^{(0)}, \quad K^{(0)} = XW_K^{(0)}.$$

For each head h , the transformation M_h defined in Section 4.1 acts within the shared head space and induces the effective scoring matrix

$$B_h := W_Q^{(0)} M_h W_K^{(0)\top}.$$

The corresponding pre-softmax attention scores can then be written equivalently as

$$S_h := \frac{XB_h X^\top}{\sqrt{d_h}} = \frac{Q^{(0)} M_h K^{(0)\top}}{\sqrt{d_h}} \in \mathbb{R}^{T \times T}.$$

The identity transformation $M = I$ recovers the base scoring matrix

$$B^{(0)} = W_Q^{(0)} W_K^{(0)\top}.$$

Allowing M to vary over $G = \text{GL}^+(d_h, \mathbb{R})$ defines the ambient scoring orbit

$$\mathcal{O}_{QK}^{(0)} = \left\{ W_Q^{(0)} M W_K^{(0)\top} : M \in G \right\}.$$

Since $M_h \in G$, every head-specific scoring matrix B_h lies on this common orbit. The generator parameterization from Section 4.1 generally parameterizes only a learned subset of the orbit rather than all of G . The shared projections determine the query and key subspaces available to every head, while M_h determines the bilinear interaction within those subspaces. The following result formalizes this structure.

Proposition 1 (Shared-subspace structure) *Suppose $W_Q^{(0)}$ and $W_K^{(0)}$ have full column rank. Then the map*

$$M \mapsto W_Q^{(0)} M W_K^{(0)\top}$$

is injective and rank-preserving, and its image consists precisely of the scoring matrices whose column space lies in $\text{col}(W_Q^{(0)})$ and whose transpose has column space lying in $\text{col}(W_K^{(0)})$. Consequently, $M_h = \exp(A_h)$ implies $\text{rank}(B_h) = d_h$.

Appendix B provides a proof. Conceptually, the proposition makes the sharing constraint explicit: the base projections fix the query and key subspaces available to every head, while M_h selects the bilinear interaction within those subspaces. Since $M_h = \exp(A_h)$ is invertible, each head under the exact exponential parameterization defines a nondegenerate, maximal-rank scoring form within the shared coordinates. Injectivity ensures that different transformations produce different effective scoring matrices, although diversity across heads is learned rather than imposed.

While S_h determines where head h attends, the value pathway determines what content it aggregates. We therefore parameterize the values using a separate shared base projection and generator bank. Let $W_V^{(0)} \in \mathbb{R}^{d_{\text{model}} \times d_h}$ and define

$$V^{(0)} = X W_V^{(0)}.$$

Let $\{E_\ell^V\}_{\ell=1}^{p_V} \subset \mathbb{R}^{d_h \times d_h}$ denote the shared value generators, and let $\theta_h^V \in \mathbb{R}^{p_V}$ denote the head-specific logits. Analogously to the query-key pathway, we define

$$\pi_h^V = \text{softmax}(\theta_h^V), \quad A_h^V = \beta_V \sum_{\ell=1}^{p_V} \pi_{h,\ell}^V E_\ell^V, \quad R_h^V = \exp(A_h^V).$$

The value representation of head h is then

$$V_h = V^{(0)} R_h^V.$$

Because the value generator bank is learned independently of the query-key generator bank, R_h^V modifies the content aggregated by each head without changing its attention weights.

Combining the head-specific values with the attention weights determined by S_h , head h produces

$$Z_h = \text{softmax}(S_h) V_h \in \mathbb{R}^{T \times d_h}.$$

The final layer output retains the standard multi-head form:

$$\text{GT-MHA}(X) = \text{Concat}(Z_1, \dots, Z_H) W_O,$$

where $W_O \in \mathbb{R}^{H d_h \times d_{\text{model}}}$. Thus, single-orbit GT-MHA leaves the standard aggregation and output-mixing operations unchanged; its structural constraint lies in generating the head-specific scoring forms and value representations from shared bases.

This construction imposes the strongest degree of sharing, but confines every head to subspaces determined by a single set of base projections. Section 4.3 relaxes this restriction by introducing multiple learned bases.

4.3 MULTI-ORBIT GT-MHA

Single-orbit GT-MHA provides the strongest form of parameter sharing, but it also imposes a common subspace structure across all heads. As established in Proposition 1, transforming a single base projection pair changes the bilinear interaction within the base query and key subspaces, but cannot change the subspaces themselves. This constraint may be overly restrictive when different heads must capture qualitatively different token interactions. Multi-orbit GT-MHA addresses this limitation by learning a small collection of base projection triples and assigning each head to one of them. Heads assigned to the same base retain the structured sharing of the single-orbit construction, while heads assigned to different bases may operate on different query, key, and value subspaces. Formally, we introduce C learned base projection triples,

$$\left\{ \left(W_Q^{(c)}, W_K^{(c)}, W_V^{(c)} \right) \right\}_{c=1}^C, \quad W_Q^{(c)}, W_K^{(c)}, W_V^{(c)} \in \mathbb{R}^{d_{\text{model}} \times d_h}.$$

We organize the H heads into C equally sized groups, with each group sharing one base projection triple. Let $c_h \in \{1, \dots, C\}$ denote the base assigned to head h . We choose C such that H is divisible by C , yielding exactly H/C heads per base. This assignment is fixed across inputs and introduces no learned routing parameters.

Applying each base projection triple to the input sequence X produces the corresponding query, key, and value representations:

$$Q^{(c)} = XW_Q^{(c)}, \quad K^{(c)} = XW_K^{(c)}, \quad V^{(c)} = XW_V^{(c)}, \quad c \in \{1, \dots, C\}.$$

A head h assigned to base c_h uses the transformation M_h from Section 4.1 to induce the effective scoring matrix

$$B_h := W_Q^{(c_h)} M_h W_K^{(c_h)\top}.$$

This scoring matrix yields the pre-softmax attention-logit matrix

$$S_h := \frac{XB_hX^\top}{\sqrt{d_h}} = \frac{Q^{(c_h)}M_hK^{(c_h)\top}}{\sqrt{d_h}} \in \mathbb{R}^{T \times T}.$$

The generator bank remains shared across all heads and bases; the multi-orbit extension changes only which learned base projections are used by each head.

Each base projection pair defines an ambient scoring orbit

$$\mathcal{O}_{QK}^{(c)} = \left\{ W_Q^{(c)} M W_K^{(c)\top} : M \in G \right\}, \quad G = \text{GL}^+(d_h, \mathbb{R}).$$

Consequently,

$$B_h \in \mathcal{O}_{QK}^{(c_h)} \quad \text{and} \quad \{B_h\}_{h=1}^H \subseteq \bigcup_{c=1}^C \mathcal{O}_{QK}^{(c)}.$$

In the single-orbit construction of Section 4.2, all head-specific scoring matrices lie within the subset of one ambient orbit realized by the learned generator parameterization. Multi-orbit GT-MHA instead distributes heads across learned subsets of several ambient orbits. For every base whose query and key projections have full column rank, Proposition 1 applies separately to its associated orbit. Consequently, transformations within an orbit vary the bilinear interaction inside a fixed pair of base subspaces, whereas different orbits may provide different query and key subspaces. This across-orbit variation supplies the additional expressivity of the multi-orbit construction.

The value pathway follows the same assignment. Using the independently learned value transformation R_h^V from Section 4.2, head h forms

$$V_h = V^{(c_h)} R_h^V, \quad Z_h = \text{softmax}(S_h) V_h.$$

The head outputs are then concatenated and mixed by W_O exactly as in the single-orbit formulation. The value generator bank, like its query–key counterpart, is shared across all bases.

5 ANALYSIS

5.1 SHARING AND EXPRESSIVITY

The number of base orbits C controls the trade-off between parameter sharing and head-specific projection capacity. When $C = 1$, multi-orbit GT-MHA reduces to the single-orbit construction, with every head derived from one base projection triple. At the other extreme, $C = H$ assigns a distinct base triple to every head and, when the head transformations are the identity, recovers the per-head projection structure of standard MHA. Intermediate values produce C orbit families, each shared by H/C heads. The regime $C \ll H$ is particularly relevant: a small collection of base subspaces supports many head-specific scoring geometries, preserving transformation-based diversity while limiting the number of independently learned subspace families.

5.2 PARAMETER COUNT

With p query-key generators and p_V value generators, and omitting biases and the unchanged output projection, GT-MHA contains

$$3C d_{\text{model}} d_h + (p + p_V) d_h^2 + H(p + p_V)$$

parameters for its base projections, generator banks, and head-specific coordinates. Standard MHA contains $3H d_{\text{model}} d_h$ query, key, and value projection parameters. A net reduction therefore occurs when

$$(p + p_V)(d_h^2 + H) < 3(H - C) d_{\text{model}} d_h.$$

The base-projection term is smaller by exactly a factor of H/C ; when $C \ll H$ and the generator overhead is lower order, the reduction in the overall attention parameter count approaches the same factor. This condition is satisfied by all configurations evaluated in Section 6, where we report the corresponding model-level counts alongside empirical performance.

5.3 COMPUTATIONAL COMPLEXITY

Relative to standard MHA, GT-MHA reduces the query, key, and value projection cost from $O(HT d_{\text{model}} d_h)$ to $O(CT d_{\text{model}} d_h)$. Constructing and applying the head-specific transformations adds $O(H(p + p_V) d_h^2)$ and $O(HT d_h^2)$ operations, respectively, while the dominant attention computation remains $O(HT^2 d_h)$. For dense transformations, exact exponentiation and the second-order approximation require $O(H d_h^3)$ operations, whereas the first-order approximation requires $O(H d_h^2)$. Only exact exponentiation guarantees membership in $\text{GL}^+(d_h, \mathbb{R})$. Appendix C provides the complete operation-level comparison and approximation details.

5.4 KV-CACHE MEMORY

During autoregressive decoding, the query for the current step is consumed immediately, whereas the keys and values of preceding tokens are retained at every layer. For batch size n , cached length T , and b bytes per scalar, standard MHA therefore requires

$$\mathcal{M}_{\text{KV}}^{\text{MHA}} = 2nTHd_h b.$$

GT-MHA need not materialize or cache H head-specific key and value histories. Instead, it retains the C base key and value streams: the head-specific query-key transformations are applied when the scores are formed, while the linear value transformations can be applied after value aggregation. Consequently,

$$\mathcal{M}_{\text{KV}}^{\text{GT}} = 2nTCd_h b = \frac{C}{H} \mathcal{M}_{\text{KV}}^{\text{MHA}}.$$

For our default $H = 16$, $C = 4$, and BF16 configuration, this reduces persistent cache storage by 75%, from 4096 to 1024 bytes per token per layer. This reduction applies specifically to the persistent KV state: GT-MHA still computes H attention maps, and temporary kernel allocations remain implementation-dependent. Appendix C.1 provides the full derivation and implementation details.

Table 1: Main comparison at matched training-token budgets. Counts are trainable parameters; attention counts aggregate all 12 layers. Lower NLL and PPL are better.

Method	Total params. (M)	Attn. params. (M)	Val. NLL	PPL
<i>TinyStoriesV2-GPT4</i> (262B Tokens)				
MHA	152.103	50.332	0.2945	1.3425
CoMHA	128.522	26.751	–	–
MQA	128.510	26.739	–	–
GQA (4 KV heads)	133.229	31.457	–	–
GT-MHA ($C = 4$, quadratic; ours)	124.581	22.810	0.2939	1.3416
<i>FineWeb-Edu</i> (10B Tokens)				
MHA	218.741	50.332	3.4018	30.0165
CoMHA	195.160	26.751	3.4799	32.3544
GT-MHA ($C = 4$, quadratic; ours)	191.219	22.810	–	–

6 EXPERIMENTS

Our experiments address four questions: (1) whether GT-MHA improves the performance–parameter trade-off relative to standard MHA; (2) how the number of base orbits C controls the trade-off between parameter sharing and predictive performance; (3) which components of the construction contribute to its effectiveness; and (4) how exact and approximate transformation modes trade predictive quality for computational efficiency.

6.1 EXPERIMENTAL SETUP

Task and data. We evaluate causal language modeling in two complementary regimes. *TinyStoriesV2-GPT4* (Eldan & Li, 2023) provides a controlled synthetic corpus: its training and validation splits contain approximately 2.23 billion and 22.5 million characters, respectively, with a shared character-level vocabulary of 230 symbols. For a broader web-text setting, we use a 10-billion-token subset of *FineWeb-Edu* (Penedo et al., 2024), comprising approximately 9.97 billion training tokens and 9.88 million validation tokens, tokenized with a 32,768-entry byte-level BPE vocabulary. All models use a context length of 512, and, within each dataset, all attention variants receive identical training and validation batches.

Architectures and baselines. For both datasets, we use a 12-layer decoder-only Transformer with $d_{\text{model}} = 1024$, $H = 16$, and $d_h = 64$, holding all non-attention components fixed. We compare GT-MHA against MHA and CoMHA (Cordonnier et al., 2020) on both corpora, and against MQA and GQA with four key–value heads on *TinyStories*. CoMHA is the closest projection-sharing baseline: it combines shared query–key projections with head-specific diagonal mixing and value projections. Unless stated otherwise, GT-MHA uses $C = 4$, quadratic transformations, transformed values, and $p = p_V = 8$.

Optimization and evaluation. We train with AdamW (learning rate 3×10^{-4} , weight decay 0.01), dropout 0.1, bfloat16 arithmetic, and an effective global batch of 2,048 sequences on four NVIDIA GH200 GPUs. All variants within a corpus share the cosine schedule and training-token budget. *FineWeb-Edu* uses 200 warmup steps and cosine decay to 3×10^{-5} over 9,512 steps, corresponding to one pass over the 9.97-billion-token subset. We report full-validation NLL and perplexity together with total and attention-specific parameter counts. Final comparisons use one fixed seed and deterministic evaluation.

6.2 MAIN COMPARISON

Table 1 compares predictive performance and parameter count at matched training-token budgets. We additionally plot validation loss against tokens processed and the performance–parameter frontier to distinguish compression from convergence behavior. Because tokenization differs across corpora, perplexity is compared only within each dataset. For *TinyStories*, we separately apply the GPT-Eval protocol of Eldan & Li (2023) to the final fixed-budget checkpoints; Appendix D provides the protocol and checkpoint-sensitivity analysis.

Table 2: Structural ablations on TinyStoriesV2-GPT4 around the default $C = 4$, $p = p_V = 8$ configuration. Lower NLL and PPL are better.

Variant	C	(p, p_V)	Value path	Total params. (M)	Attn params. (M)	Val. NLL	PPL
Default GT-MHA	4	(8,8)	Transformed	124.581	22.810	–	–
Single orbit	1	(8,8)	Transformed	117.503	15.732	–	–
Eight orbits	8	(8,8)	Transformed	134.018	32.247	–	–
Per-head bases	16	(8,8)	Transformed	152.893	51.121	–	–
Fewer generators	4	(4,4)	Transformed	124.186	22.415	–	–
More generators	4	(16,16)	Transformed	125.371	23.599	–	–
Shared values	4	(8,–)	Untransformed	124.186	22.415	–	–

Table 3: Transformation-mode ablation on TinyStoriesV2-GPT4. MHA is included as a reference; all GT-MHA variants use $C = 4$ and $p = p_V = 8$. Attention GFLOPs per sequence denotes the attention-module forward cost at context length 512. Throughput and memory are measured in BF16 under matched hardware and batch settings. Lower PPL, FLOPs, GPU-hours, and memory are better; higher throughput is better.

Method	Val. PPL	Attn. GFLOPs/seq.	Train tok./s/GPU	GPU-h	Train peak (GB)	Prefill tok./s	Decode tok./s	KV B/tok./layer
MHA (reference)	–	–	–	–	–	–	–	4096
GT-MHA (exact exponential)	–	–	–	–	–	–	–	1024
GT-MHA (second-order; quadratic)	–	–	–	–	–	–	–	1024
GT-MHA (first-order; residual)	–	–	–	–	–	–	–	1024

6.3 STRUCTURAL ABLATIONS

Structural configuration. Table 2 isolates the effects of the number of bases, generator-bank size, and value transformations under otherwise matched architecture, optimization, and training-token settings.

Transformation approximation and efficiency. Table 3 compare the exact matrix exponential with its first- and second-order Taylor approximations, corresponding to the exponential, residual, and quadratic GT-MHA variants. For each mode, we report validation perplexity, training throughput in tokens per second per GPU, GPU-hours, and peak accelerator memory. At inference, we benchmark a cache-producing prefill followed by single-token KV-cached decoding at matched batch size, precision, prompt length, and generation length. We report prefill and decoding throughput, measured persistent KV-cache bytes, and peak memory, and verify the measured cache size against the analysis in Section 5.4. These measurements complement the asymptotic comparison in Appendix C and quantify the practical quality–efficiency trade-off.

7 LIMITATIONS

GT-MHA introduces a structured sharing bias: heads are generated from a finite set of base projections rather than parameterized independently. This design creates an explicit efficiency–expressivity trade-off controlled by C ; tasks benefiting from more heterogeneous head sub-spaces may favor larger C . Our current implementation uses fixed head-to-base assignments and a generator bank shared across heads, while input-dependent routing represents a natural extension. Exact matrix exponentiation also introduces additional computation, although the first- and second-order approximations studied in Section 5 provide lower-cost alternatives. Empirically, we focus on 12-layer decoder-only language models trained on TinyStoriesV2 and FineWeb-Edu. Because the main comparisons use a single training seed, they do not characterize run-to-run variance. Future evaluation at larger scales, across additional architectures and modalities, and with multiple seeds would further test the generality of the approach.

GT-MHA has lower parameter count, projection FLOPs, and parameter-memory traffic than MHA, and its residual formulation has the potential for higher throughput with specialized fused kernels.

8 CONCLUSION

We introduced group-transformed multi-head attention (GT-MHA), which generates head-specific scoring geometries by applying learned Lie-group transformations to one or more base projections. The single-orbit construction maximizes parameter sharing, while the multi-orbit extension allows different groups of heads to operate in distinct learned subspaces. Our analysis characterizes the resulting sharing structure and its parameter and computational costs, and our experimental framework compares GT-MHA with standard and projection-sharing attention baselines under matched training budgets. More broadly, GT-MHA provides a principled mechanism for trading unconstrained head independence for structured diversity while retaining the standard attention interface.

REFERENCES

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebr’on, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 4895–4901, 2023.
- Srinadh Bhojanapalli, Chulhee Yun, Ankit Singh Rawat, Sashank J. Reddi, and Sanjiv Kumar. Low-rank bottleneck in multi-head attention models. In *Proceedings of the 37th International Conference on Machine Learning*, pp. 864–873, 2020.
- Yilong Chen, Linhao Zhang, Junyuan Shang, Zhenyu Zhang, Tingwen Liu, Shuohuan Wang, and Yu Sun. Dha: Learning decoupled-head attention from transformer checkpoints via adaptive heads fusion. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- Jean-Baptiste Cordonnier, Andreas Loukas, and Martin Jaggi. Multi-head attention: Collaborate instead of concatenate. *arXiv preprint arXiv:2006.16362*, 2020.
- DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.
- Ronen Eldan and Yuanzhi Li. Tinstories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.
- Marc Finzi, Samuel Stanton, Pavel Izmailov, and Andrew Gordon Wilson. Generalizing convolutional neural networks for equivariance to lie groups on arbitrary continuous data. In *Proceedings of the 37th International Conference on Machine Learning*, pp. 3165–3176, 2020.
- Michael J. Hutchinson, Charline Le Lan, Shehryar Zaidi, Emilien Dupont, Yee Whye Teh, and Hyunjik Kim. Lietransformer: Equivariant self-attention for lie groups. In *Proceedings of the 38th International Conference on Machine Learning*, pp. 4533–4543, 2021.
- Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Tan Nguyen, Tam Nguyen, Hai Do, Khai Nguyen, Vishwanath Saragadam, Minh Pham, Khuong Duy Nguyen, Nhat Ho, and Stanley Osher. Improving transformer with an admixture of attention heads. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- Guilherme Penedo, Hynek Kydlicek, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems*, 2024.
- David W. Romero and Jean-Baptiste Cordonnier. Group equivariant stand-alone self-attention for vision. In *International Conference on Learning Representations*, 2021.
- Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 5797–5808, 2019.

Huiyin Xue and Nikolaos Aletras. Pit one against many: Leveraging attention-head embeddings for parameter-efficient multi-head attention. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 10355–10373, 2023.

A LARGE LANGUAGE MODEL USAGE

OpenAI Codex was used for language editing and organizational suggestions during manuscript preparation, literature discovery, and limited code implementation and debugging assistance. The research ideas, method, theoretical development, experimental design, results, and conclusions are the authors’ work. All LLM-assisted content was reviewed and verified by the authors, who take full responsibility for the paper.

B PROOF OF PROPOSITION 1

Proof. Let $U = W_Q^{(0)}$ and $V = W_K^{(0)}$. Because U and V have full column rank, their left pseudoinverses

$$U^\dagger = (U^\top U)^{-1} U^\top, \quad V^\dagger = (V^\top V)^{-1} V^\top$$

satisfy $U^\dagger U = V^\dagger V = I_{d_h}$. For any $M \in \mathbb{R}^{d_h \times d_h}$,

$$\text{col}(UMV^\top) \subseteq \text{col}(U), \quad \text{col}((UMV^\top)^\top) \subseteq \text{col}(V),$$

so every matrix in the image satisfies the stated subspace constraints.

Conversely, let $B \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ satisfy

$$\text{col}(B) \subseteq \text{col}(U), \quad \text{col}(B^\top) \subseteq \text{col}(V).$$

Define $P_U = UU^\dagger$ and $P_V = VV^\dagger$. These conditions imply $P_U B = B$ and $B P_V = B$. Setting

$$M = U^\dagger B (V^\dagger)^\top$$

therefore gives

$$UMV^\top = P_U B P_V = B.$$

Hence, the image is precisely the claimed family of scoring matrices.

Moreover,

$$U^\dagger (UMV^\top) (V^\dagger)^\top = M.$$

Thus $M \mapsto UMV^\top$ is injective. The general rank inequality gives $\text{rank}(UMV^\top) \leq \text{rank}(M)$, while the displayed recovery of M gives the reverse inequality. Therefore,

$$\text{rank}(UMV^\top) = \text{rank}(M).$$

Finally, $M_h = \exp(A_h)$ is invertible, so $\text{rank}(M_h) = d_h$. Taking $M = M_h$ yields $\text{rank}(B_h) = d_h$. ■

C DETAILED COMPUTATIONAL COMPLEXITY

This section expands the computational summary in Section 5. Table 4 separates the per-layer forward-pass costs of standard MHA and the three GT-MHA transformation modes. We suppress the batch dimension and constant factors, and assume dense generators with the same H , d_h , and d_{model} across variants.

GT-MHA replaces the H query, key, and value projection triples of standard MHA with C base triples. Forming the head-specific Lie-algebra elements from the shared generator banks costs $O(H(p + p_V)d_h^2)$, and applying the query–key and value transformations costs $O(HTd_h^2)$. The attention-score computation and value aggregation remain $O(HT^2d_h)$.

Table 4: Per-layer forward-pass complexity of standard MHA and GT-MHA. The first- and second-order columns use truncated Taylor approximations of the matrix exponential.

Operation	MHA	First order	Second order	Exact
Base Q/K/V projections	$O(HTd_{\text{model}}d_h)$	$O(CTd_{\text{model}}d_h)$	$O(CTd_{\text{model}}d_h)$	$O(CTd_{\text{model}}d_h)$
Generator combination	–	$O(H(p + p_V)d_h^2)$	$O(H(p + p_V)d_h^2)$	$O(H(p + p_V)d_h^2)$
Transformation evaluation	–	$O(Hd_h^2)$	$O(Hd_h^3)$	$O(Hd_h^3)$
Applying transformations	–	$O(HTd_h^2)$	$O(HTd_h^2)$	$O(HTd_h^2)$
Attention and aggregation	$O(HT^2d_h)$	$O(HT^2d_h)$	$O(HT^2d_h)$	$O(HT^2d_h)$

The transformation-evaluation cost depends on how the exponential map is computed. In addition to the exact construction $M_h = \exp(A_h)$, we consider

$$\widetilde{M}_h^{(1)} = I + A_h, \quad \widetilde{M}_h^{(2)} = I + A_h + \frac{1}{2}A_h^2.$$

We apply the same truncations to the value transformation R_h^V . Once A_h has been formed, the first-order approximation costs $O(Hd_h^2)$, whereas the second-order approximation requires a dense matrix multiplication and costs $O(Hd_h^3)$. Exact dense matrix exponentiation also costs $O(Hd_h^3)$ asymptotically, although its wall-clock cost can differ from the second-order approximation because of algorithmic constants. All three variants preserve the dominant $O(HT^2d_h)$ attention term.

Only the exact exponential guarantees that the resulting transformations lie in $\text{GL}^+(d_h, \mathbb{R})$. The truncated Taylor variants are not guaranteed to remain invertible and are therefore computational approximations rather than exact Lie-group parameterizations. Their practical latency and throughput are evaluated empirically in Section 6.

C.1 BASE-LEVEL KV CACHING

Consider autoregressive decoding after a prefix of T tokens for batch size n . Standard MHA stores, at each layer,

$$K_{\text{MHA}}, V_{\text{MHA}} \in \mathbb{R}^{n \times H \times T \times d_h},$$

and therefore requires $2nTHd_hb$ bytes when each scalar occupies b bytes.

For GT-MHA, let $K^{(c)}, V^{(c)} \in \mathbb{R}^{n \times T \times d_h}$ be the representations produced by base c . The score for a new query uses the cached base key directly,

$$S_h = \frac{(q^{(c_h)} M_h) K^{(c_h)\top}}{\sqrt{d_h}},$$

so no head-specific key history needs to be stored. For the value path, linearity gives

$$\text{softmax}(S_h)(V^{(c_h)} R_h^V) = (\text{softmax}(S_h) V^{(c_h)}) R_h^V.$$

Thus R_h^V may be applied after aggregation, and the persistent cache contains only

$$K_{\text{GT}}, V_{\text{GT}} \in \mathbb{R}^{n \times C \times T \times d_h}.$$

Its size is $2nTCd_hb$, a fraction C/H of the MHA cache. More generally, if key and value widths differ, the size is $nTC(d_k + d_v)b$.

Our implementation stores these base tensors separately at each layer and appends one base key and value vector per generated token. The inference benchmark measures the returned cache tensors directly and compares them with the analytical count. This result concerns persistent cache storage: kernels without native grouped-query support may transiently expand base tensors for the attention operation, and attention-map computation still scales with H .

D TINYSTORIES GPT-EVAL PROTOCOL AND EXAMPLES

Checkpoint selection. For each method in the main comparison, the primary GPT-Eval scores use the final checkpoint obtained after the matched training-token budget. We additionally evaluate the checkpoint with the lowest validation negative log-likelihood, selected without reference to GPT-Eval. If the two selection rules identify the same checkpoint, it is evaluated only once.

Generation protocol. All methods receive the same fixed set of story prefixes and use identical decoding hyperparameters, completion lengths, and generation seeds. We sample the same number of completions for every prompt and method. Model identities are removed and the completions are randomly ordered before grading.

Grading and aggregation. Following Eldan & Li (2023), the judge scores each completion from 1 to 10 for grammar, creativity, and consistency with the supplied story prefix. We report averages over the fixed prompt set together with 95% bootstrap confidence intervals computed across prompts. The final submission will specify the exact judge-model identifier, grading prompt, decoding configuration, number of prompts, and number of completions per prompt.

Table 5: Sensitivity of TinyStories GPT-Eval scores to checkpoint selection. The best checkpoint is selected solely by validation NLL. Scores are reported as mean with 95% bootstrap confidence intervals across prompts; higher is better.

Method	Checkpoint	Grammar	Creativity	Consistency
MHA	Final	–	–	–
MHA	Best validation	–	–	–
CoMHA	Final	–	–	–
CoMHA	Best validation	–	–	–
GT-MHA	Final	–	–	–
GT-MHA	Best validation	–	–	–

Evaluation materials. The final appendix will include the complete prompt set, the grading prompt, and representative anonymized completions from each method. All generated completions and raw judge responses will be released as machine-readable supplementary files.